

SISTEMA INTEGRADOR

Guia Técnico de Telas e Funcionalidades

Mapeamento: elemento da interface → rota → classe/método Java responsável

Projeto: integrador | Stack: Java (Servlet/JSP, Maven, Tomcat, MySQL)

1. Visão Geral do Sistema

O "integrador" é um sistema web para gestão de restaurante, construído em Java puro (Servlets + JSP), sem frameworks como Spring ou Struts. A arquitetura segue o padrão MVC feito "na mão":

- Model (M): classes em `br.com.restaurante.model` — representam as tabelas do banco (Usuario, Cardapio, Mesa, Pedido, ItemPedido, Pagamento, FilaPreparo, CategoriaItem).
- View (V): páginas JSP em `src/main/webapp/WEB-INF/views/` — cada pasta corresponde a uma área do sistema (auth, dashboard, cardapio, mesa, pedido, fila, relatorio, staff, cliente).
- Controller (C): Servlets em `br.com.restaurante.controller` — cada um mapeado a uma rota fixa via `@WebServlet`, e que decide qual JSP mostrar (GET) ou qual ação executar (POST).
- DAO: classes em `br.com.restaurante.dao` — fazem o acesso direto ao banco (SQL) para cada tabela, chamadas pelos Controllers.

Como ler este documento

Cada seção a seguir corresponde a UMA TELA (ou grupo de telas) do sistema. A tabela de cada seção mostra, para cada botão/link/formulário importante da tela:

- Elemento na tela — o botão, link ou formulário que o usuário vê e clica.
- Ação/evento — o que acontece ao clicar (ex.: envia formulário POST, chama função JS, navega para outra URL).
- Rota (HTTP) — a URL/parâmetro que é efetivamente chamado no servidor.
- Classe.método (Java) — o Controller e o método interno que trata aquela rota/ação.
- O que faz — resumo da lógica executada (validações, DAO chamado, para onde redireciona).

Fluxo geral de navegação por perfil

O sistema tem 3 perfis de usuário (tabela `usuario`, coluna `perfil`), controlados no login por `AuthController.destino()`:

- GERENTE → cai em `/app/dashboard` (visão gerencial completa: dashboard, mesas, cardápio, pedidos, fila, relatórios, staff).
- FUNCIONARIO (atendente ou cozinha, tratados de forma unificada) → cai em `/app/mesas` (mesas, cardápio, pedidos, fila).
- USUARIO (cliente final do app) → cai em `/app/cardapio` (cardápio, pedido de mesa, delivery, reserva, meus pedidos).

O controle de sessão/URL é reforçado pelo `AuthFilter` (`br.com.restaurante.filter.AuthFilter`), que intercepta toda rota `/app/*` e barra quem não está logado, redirecionando para `/auth/login`. Cada Controller, além disso, checka o perfil especificamente (`isGerente()`, `temPermissaoFuncionario()`, `isCliente()`, etc.) antes de executar qualquer ação sensível.

Mapa de rotas (visão rápida)

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Login / Cadastro / Logout	GET/POST	<code>/auth/login</code> , <code>/auth/cadastro</code> , <code>/auth/logout</code>	<code>AuthController</code>	Autenticação e criação de conta
Dashboard gerencial	GET	<code>/app/dashboard</code>	<code>DashboardController</code>	Painel com métricas (só GERENTE)
Cardápio (CRUD de itens)	GET/POST	<code>/app/cardapio</code>	<code>CardapioController</code>	Listar/criar/editar/excluir itens e categorias
Mesas do salão	GET/POST	<code>/app/mesas</code>	<code>MesaController</code>	Listar, abrir, fechar, reservar, CRUD de mesas
Pedidos (mesa/delivery interno)	GET/POST	<code>/app/pedidos</code>	<code>PedidoController</code>	Criar pedido, avançar status, cancelar, pagamento

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Fila de preparo	GET/POST	/app/fila	FilaController	Ver fila por setor, assumir e concluir preparo
Relatórios em PDF	GET/POST	/app/relatorios	RelatorioController	Gerar PDF de faturamento por período
Gestão de funcionários	GET/POST	/app/staff	StaffController	Cadastrar/desativar contas de FUNCIONARIO/GERENTE
Área do cliente (delivery/reserva/mesa)	GET/POST	/app/cliente/*	ClienteController	Pedido delivery, reserva de mesa, pedido direto na mesa, meus pedidos
Imagens do cardápio	GET	/imagens/cardapio/*	ImagemServlet	Serve as fotos dos itens (fora da pasta do projeto)

2. Autenticação (Login / Cadastro / Logout)

2.1 Tela de Login

Arquivo(s) JSP: WEB-INF/views/auth/login.jsp

Controller: AuthController (br.com.restaurante.controller.AuthController)

DAO(s) envolvidas: UsuarioDAO

Quem pode acessar: Pública (sem login)

Formulário simples de autenticação. Se o usuário já estiver logado e acessar esta URL, é redirecionado direto para a tela do seu perfil (método destino()).

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Campos login e senha	Digitação	—	—	Apenas campos de texto do <form>.
Botão "Entrar" (submit)	Envia <form method="POST" action=".../auth/login">	POST /auth/login	AuthController.processarLogin()	Busca usuário por login (UsuarioDAO.buscarPorLogin), confere senha com BCrypt.checkpw(); se OK cria sessão (usuarioLogado) e chama destino(u) para redirecionar (GERENTE→dashboard, FUNCIONARIO→mesas, USUARIO→cardapio); se erro, volta para login.jsp com mensagem genérica.
Link "Criar conta"	Navegação <a href>	GET /auth/cadastro	AuthController.exibirCadastro()	Encaminha para cadastro.jsp (ou redireciona se já logado).

2.2 Tela de Cadastro

Arquivo(s) JSP: WEB-INF/views/auth/cadastro.jsp

Controller: AuthController

DAO(s) envolvidas: UsuarioDAO

Quem pode acessar: Pública (sem login) — sempre cria conta com perfil USUARIO (cliente)

Autocadastro público. Diferente da tela de Staff (seção 9), aqui o perfil é sempre forçado para USUARIO — nunca GERENTE/FUNCIONARIO.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Campos nome, login, senha, confirmarSenha	Digitação + validação JS (função validar())	—	—	Validação client-side antes de liberar o submit (ex.: senhas coincidem).
Botão "Criar conta" (submit)	Envia <form method="POST" action=".../auth/cadastro">	POST /auth/cadastro	AuthController.processarCadastro()	Valida campos no servidor (nome, login ≥3 caracteres único, senha ≥6, confirmação), gera hash BCrypt, insere via UsuarioDAO.inserir() sempre com perfil="USUARIO", grava mensagem de sucesso na

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
				sessão e redireciona para /auth/login.
Link "Faça login"	Navegação <a href>	GET /auth/login	AuthController.exibirLogin()	Volta para a tela de login.

2.3 Logout

Arquivo(s) JSP: (sem JSP própria – link no menu lateral _sidebar.jsp)

Controller: AuthController

DAO(s) envolvidas: –

Quem pode acessar: Qualquer usuário logado

Encerramento de sessão, disponível no menu lateral em todas as telas internas.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Link "Sair" (menu lateral, _sidebar.jsp)	Navegação <a href>	GET /auth/logout	AuthController.logout()	Invalida a HttpSession (s.invalidate()) e redireciona para /auth/login.

3. Dashboard Gerencial

3.1 Painel do Gerente

Arquivo(s) JSP: WEB-INF/views/dashboard/dashboard.jsp

Controller: DashboardController

DAO(s) envolvidas: MesaDAO, PedidoDAO, ItemPedidoDAO (consultas SQL diretas dentro do próprio controller para os gráficos)

Quem pode acessar: Exclusivo perfil GERENTE (qualquer outro perfil é redirecionado para /app/mesas)

Tela sem ações de escrita — é só leitura/visualização. Todo o conteúdo (contagem de mesas, pedidos abertos, faturamento do dia, gráfico de receita dos últimos 7 dias e gráfico de vendas por setor) é montado no doGet() e passado para a JSP via request.setAttribute(). Os gráficos são SVG desenhados na própria JSP a partir desses dados (sem biblioteca JS).

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Cartões de contagem de mesas (livres/ocupadas/reservadas)	Carregamento da página	GET /app/dashboard	DashboardController.doGet() — STEP 2	MesaDAO.listar() + contagem por status (stream/filter) em Java.
Lista de pedidos em aberto	Carregamento da página	GET /app/dashboard	DashboardController.doGet() — STEP 3	PedidoDAO.listarAbertos(); exibe só os 8 mais recentes.
Cartão "Faturamento de hoje" / "Ticket médio"	Carregamento da página	GET /app/dashboard	DashboardController.buscarTotalHoje()	SQL somando pedidos entregues com data de hoje; ticket médio = total ÷ nº de pedidos entregues hoje.
Gráfico "Receita da Semana" (linha)	Carregamento da página	GET /app/dashboard	DashboardController.buscarReceitaUltimos7Dias()	SQL agrupando faturamento por dia nos últimos 7 dias; devolve um Map dia→valor desenhado como SVG na JSP.
Gráfico "Vendas por Categoria" (rosca)	Carregamento da página	GET /app/dashboard	DashboardController.buscarVendasPorSetor()	SQL com JOIN item_pedido→cardapio→categoria_item, somando vendas por setor nos últimos 7 dias.
Link "Ver todos os pedidos"	Navegação <a href>	GET /app/pedidos	PedidoController.listar()	Leva para a tela completa de Pedidos (seção 6).

4. Cardápio (CRUD de Itens)

4.1 Listagem do Cardápio

Arquivo(s) JSP: WEB-INF/views/cardapio/cardapio.jsp

Controller: CardapioController

DAO(s) envolvidas: CardapioDAO, CategoriaItemDAO

Quem pode acessar: Listagem: qualquer perfil logado. Escrita (criar/editar/excluir/disponibilidade/categorias): só GERENTE.

Também é a TELA INICIAL do cliente (perfil USUARIO) após o login. Uma única rota /app/cardapio é compartilhada por todos os perfis, roteada pelo parâmetro "acao".

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Pills de filtro ("Todos", "Pratos", "Bebidas", "Sobremesas")	onclick="filtrarSetor (...) "	– (filtro 100% client-side, JS)	função JS filtrarSetor()	Só esconde/mostra cards já carregados na página; não faz nova requisição ao servidor.
Botão "+ Novo Item"	Navegação <a href>	GET /app/cardapio?acao=novo	CardapioController.exibirFormularioNovo()	Checa isGerente(); monta form_item.jsp em branco, com a lista de categorias para o <select>.
Ícone lápis (editar item)	Navegação <a href>	GET /app/cardapio?acao=editar&id=X	CardapioController.exibirFormularioEditar()	Checa isGerente(); busca o item via CardapioDAO.buscarPorId(id) e preenche form_item.jsp.
Ícone lixeira (excluir item)	Envia mini- <code><form method="POST"></code> oculto com acao=excluir	POST /app/cardapio (acao=excluir)	CardapioController.excluir()	CardapioDAO.desativar(id) — soft delete (ativo=0), preserva histórico de pedidos antigos.
Toggle de disponibilidade do item	JS monta e envia #formDisponivel com acao=disponivel	POST /app/cardapio (acao=disponivel)	CardapioController.atualizarDisponibilidade()	CardapioDAO.atualizarDisponibilidade(id, valor) — atualiza só a coluna "disponivel", sem tocar no resto do item.
Botão "⚙️ Categorias" (abre modal)	onclick="abrirModalCategorias() "	GET /app/cardapio (AJAX, mesma rota, ação de listagem de categorias)	CardapioController — bloco de categorias	CategoriaItemDAO.listar() alimenta o modal.
Botão "+ Adicionar" (nova categoria, dentro do modal)	JS envia POST assíncrono	POST /app/cardapio (acao=criarCategoria)	CardapioController.criarCategoria()	CategoriaItemDAO.inserir() — cria nova linha em categoria_item (nome + setor).
Ícone lápis na linha de categoria (renomear)	onclick="editarLinhaCategoria(this)" + "salvarLinhaCategoria(this)"	POST /app/cardapio (acao=editarCategoria)	CardapioController.editarCategoria()	CategoriaItemDAO.editar() — atualiza nome/setor da categoria.
Ícone lixeira na linha de categoria (excluir)	onclick="excluirCategoria(id,this)"	POST /app/cardapio (acao=excluirCategoria)	CardapioController.excluirCategoria()	CategoriaItemDAO.desativar(id) — soft delete da categoria.

4.2 Formulário de Item (Novo/Editar)

Arquivo(s) JSP: WEB-INF/views/cardapio/form_item.jsp

Controller: CardapioController

DAO(s) envolvidas: CardapioDAO, CategoriaItemDAO, UploadImagemUtil

Quem pode acessar: Exclusivo GERENTE

Um único formulário serve tanto para criar quanto para editar (diferenciado pelo campo oculto id: id<=0 é novo, id>0 é edição). Usa multipart/form-data por causa do upload de imagem.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Campos nome, categoria, preço, tempo de preparo, descrição, imagem	Digitação / upload de arquivo	—	—	Campos do <form enctype="multipart/form-data">.
Link "+ nova categoria" (abre modal dentro do form)	onclick="abrirModalCategoria()"	AJAX interno	CardapioController.criarCategoria() (mesmo endpoint da seção 4.1)	Permite criar categoria sem sair da tela de cadastro do item.
Botão "Salvar" (submit)	Envia <form method="POST" action="/app/cardapio"> com acao=salvar	POST /app/cardapio (acao=salvar)	CardapioController.salvar()	Converte preço (aceita vírgula), processa o Part "imagem" via UploadImagemUtil.salvar() (limite 3 MB, grava a imagem em pasta externa ao projeto), e chama CardapioDAO.inserir() (id≤0) ou CardapioDAO.editar() (id>0). Redireciona para a listagem com mensagem de sucesso.
Link "Cancelar"	Navegação <a href>	GET /app/cardapio	CardapioController.listar()	Volta para a listagem sem salvar.

5. Mesas do Salão

5.1 Listagem de Mesas

Arquivo(s) JSP: WEB-INF/views/mesa/mesas.jsp

Controller: MesaController

DAO(s) envolvidas: MesaDAO

Quem pode acessar: GERENTE e FUNCIONARIO (USUARIO usa a área de cliente — seção 8)

Mostra todas as mesas com seu status (livre/ocupada/reservada) e permite operar sobre elas.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Botão "Abrir" / marcar ocupada (por mesa)	onclick que preenche #modalAcao e envia form	POST /app/mesas (acao=abrirMesa)	MesaController.mudarStatus(..., "ocupada")	MesaDAO.atualizarStatus(id, "ocupada", operador) — grava também quem operou e a data/hora.
Botão "Fechar" / liberar mesa	idem (modal com acao=fecharMesa)	POST /app/mesas (acao=fecharMesa)	MesaController.mudarStatus(..., "livre")	MesaDAO.atualizarStatus(id, "livre", operador).
Botão "Atender chamado" (quando cliente chamou o garçom pela mesa)	Envia <form> oculto com acao=atenderChamado	POST /app/mesas (acao=atenderChamado)	MesaController.atenderChamado()	MesaDAO.atenderChamado(id Mesa) — limpa o sinalizador de chamado da mesa.
Ícone lixeira (excluir mesa)	Envia <form> oculto com acao=excluir	POST /app/mesas (acao=excluir)	MesaController — bloco salvar/excluir	MesaDAO.desativar(id) — soft delete.
Botão "+ Nova mesa" (link para form_mesa.jsp)	Navegação <a href>	GET /app/mesas?acao=novo	MesaController.doGet() — bloco CRUD (exige isGerente())	Abre form_mesa.jsp em branco (número/capacidade da mesa).
Clique no card da mesa (ver detalhe)	Navegação <a href>	GET /app/mesas?acao=detalhe&id=X	MesaController.exibirDetalhe()	MesaDAO.buscarPorId(id) + PedidoDAO (pedidos abertos daquela mesa) → mesa/detalhe_mesa.jsp.

5.2 Formulário de Mesa (Novo/Editar)

Arquivo(s) JSP: WEB-INF/views/mesa/form_mesa.jsp

Controller: MesaController

DAO(s) envolvidas: MesaDAO

Quem pode acessar: Exclusivo GERENTE

CRUD administrativo simples: número e capacidade da mesa.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Botão "Criar mesa" / "Salvar" (submit)	Envia <form method="POST" action=".../app/mesas"> com acao=salvar	POST /app/mesas (acao=salvar)	MesaController — bloco salvar/excluir	MesaDAO.inserir() (nova) ou MesaDAO.editar() (id>0 existente).
Link "Cancelar" / "← Voltar"	Navegação <a href>	GET /app/mesas	MesaController.listarMesas()	Volta para a listagem sem salvar.

5.3 Detalhe da Mesa

Arquivo(s) JSP: WEB-INF/views/mesa/detalhe_mesa.jsp

Controller: MesaController + PedidoController

DAO(s) envolvidas: MesaDAO, PedidoDAO

Quem pode acessar: GERENTE e FUNCIONARIO

Mostra os pedidos abertos vinculados àquela mesa específica.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Link de cada pedido listado	Navegação <a href>	GET /app/pedidos?acao=detalhe&id=X	PedidoController.exibirDetalhe()	Abre o detalhe daquele pedido (seção 6.3).
Botão "Novo pedido nesta mesa"	Navegação <a href>	GET /app/pedidos?acao=novo&mesaId=X	PedidoController.exibirFormulario()	Abre novo_pedido.jsp já com a mesa pré-selecionada.
Link "← Voltar"	Navegação <a href>	GET /app/mesas	MesaController.listarMesas()	Volta para a listagem de mesas.

6. Pedidos (Mesa e Delivery interno)

6.1 Listagem de Pedidos

Arquivo(s) JSP: WEB-INF/views/pedido/pedidos.jsp

Controller: PedidoController

DAO(s) envolvidas: PedidoDAO, ItemPedidoDAO, PagamentoDAO

Quem pode acessar: GERENTE e FUNCIONARIO

Usado pela equipe para criar, acompanhar e finalizar pedidos feitos na mesa ou lançados manualmente como delivery.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Pills de filtro por status (Todos/Recebido/Em Preparo/Pronto/Entregue/Cancelado)	onclick="filtrarStatus(...)"	– (filtro client-side, JS)	função JS <code>filtrarStatus()</code>	Filtra os cards já renderizados, sem nova requisição.
Botão "+ Novo Pedido"	Navegação <a href>	GET /app/pedidos?acao=novo	<code>PedidoController.exibirFormulario()</code>	<code>MesaDAO.listarLivres()</code> + <code>CardapioDAO.listar()</code> → monta <code>novo_pedido.jsp</code> .
Ícone "Avançar status" (por pedido)	onclick abre modal #modalStatus, form envia <code>acao=avancarStatus</code>	POST /app/pedidos (acao=avancarStatus)	<code>PedidoController.avancarStatus()</code>	Máquina de estados aberto→em_preparo→pronto→entregue; ao virar em_preparo também inicia registro na fila (<code>FilaPreparoDAO</code>).
Ícone "Registrar entrega" (quando status=pronto)	onclick abre modal #modalEntrega (com formas de pagamento)	POST /app/pedidos (acao=avancarStatus, com <code>formaPagamento/valorPagamento</code>)	<code>PedidoController.avancarStatus()</code>	Conclui a fila (<code>FilaPreparoDAO.concluir()</code>), muda status para "entregue" e registra o Pagamento (<code>PagamentoDAO.registrar</code>) — valor informado ou calculado automaticamente somando os itens.
Botão "+ Adicionar forma de pagamento" (dentro do modal de entrega)	onclick="adicionarLinhaPagamento()"	– (JS, só monta campos extras no formulário)	função JS <code>adicionarLinhaPagamento()</code>	Permite split de pagamento (várias formas para o mesmo pedido) antes de enviar o POST.
Ícone "Cancelar" (X)	Envia <form> oculto com <code>acao=cancelar</code>	POST /app/pedidos (acao=cancelar)	<code>PedidoController.cancelar()</code>	<code>PedidoDAO.desativar()</code> + <code>ItemPedidoDAO.cancelarItensDoPedido()</code> + <code>FilaPreparoDAO.desativar()</code> — tudo em uma transação (rollback total se algo falhar).
Clique no card do pedido	Navegação <a href>	GET /app/pedidos?acao=detalhe&id=X	<code>PedidoController.exibirDetalhe()</code>	Abre o detalhe completo do pedido (seção 6.3).

6.2 Novo Pedido

Arquivo(s) JSP: WEB-INF/views/pedido/novo_pedido.jsp

Controller: PedidoController

DAO(s) envolvidas: MesaDAO, CardapioDAO, ItemPedidoDAO, FilaPreparoDAO, PedidoDAO

Quem pode acessar: GERENTE e FUNCIONARIO

Formulário para montar um pedido do zero, escolhendo tipo (mesa ou delivery), mesa (se aplicável), itens do cardápio e quantidades.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Select "Tipo" (Mesa/Delivery)	onchange (JS mostra/esconde campo de mesa)	—	—	Só altera a exibição do formulário.
Pills de filtro de itens por setor	onclick	— (JS, filtra cards de itens já carregados)	—	Não requisita o servidor.
Botões +/- de quantidade em cada item	onclick="aumentar()" / "diminuir()"	— (JS, atualiza campos ocultos itemId[]/quantidade[])	—	Monta as listas paralelas que serão enviadas no submit.
Botão "Confirmar Pedido" (submit)	Envia <form method="POST" action=".../app/pedidos"> com acao=criar	POST /app/pedidos (acao=criar)	PedidoController.criar()	Cria o Pedido "casca" (PedidoDAO.inserir), valida cada item no CardapioDAO (usa sempre o preço atual do banco, nunca o do formulário), insere os itens em lote (ItemPedidoDAO.inserirLote) e cria a entrada na fila (FilaPreparoDAO.inserir) com peso calculado por Pedido.calcularPeso() (considera a flag "urgente"). Tudo em uma única transação.
Link "← Voltar" / "Cancelar"	Navegação <a href>	GET /app/pedidos	PedidoController.listar()	Volta para a listagem sem criar nada.

6.3 Detalhe do Pedido

Arquivo(s) JSP: WEB-INF/views/pedido/detalhe_pedido.jsp

Controller: PedidoController

DAO(s) envolvidas: PedidoDAO, ItemPedidoDAO, PagamentoDAO

Quem pode acessar: GERENTE e FUNCIONARIO

Mostra os itens do pedido, pagamentos já registrados e permite avançar o status ou cancelar.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Botão "Avançar status" (ex.: aberto→em_preparo)	Envia <form method="POST"> com acao=avancarStatus	POST /app/pedidos (acao=avancarStatus)	PedidoController.avancarStatus()	Mesma lógica da listagem (seção 6.1) — máquina de estados.
Botão "Confirmar entrega" (abre modal de pagamento quando status=pronto)	onclick="abrirModalPagamento()" + submit do form com acao=avancarStatus	POST /app/pedidos (acao=avancarStatus, com dados de pagamento)	PedidoController.avancarStatus()	Registra Pagamento(s) e marca como "entregue", igual à seção 6.1.
Botão "X Cancelar pedido"	Envia <form method="POST"> com acao=cancelar	POST /app/pedidos (acao=cancelar)	PedidoController.cancelar()	Soft delete em cascata do pedido + itens + fila (mesma lógica da seção 6.1).

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Link "← Voltar"	Navegação <a href>	GET /app/pedidos	PedidoController.listar()	Retorna para a listagem.

7. Fila de Preparo

7.1 Painel da Fila (Cozinha / Bebida / Sobremesa)

Arquivo(s) JSP: WEB-INF/views/fila/fila.jsp e fila/_card_fila.jsp (fragmento reutilizado por card de pedido)

Controller: FilaController

DAO(s) envolvidas: FilaPreparoDAO, PedidoDAO

Quem pode acessar: GERENTE e FUNCIONARIO (qualquer função — atendente ou cozinha)

Três colunas/painéis (um por setor), cada um listando os pedidos pendentes daquele setor.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Botão "Assumir" (em cada card de pedido da fila)	onclick abre modal com acao=iniciar pré-selecionada	POST /app/fila (acao=iniciar)	FilaController.iniciarPreparo()	FilaPreparoDAO.iniciarPreparo(id, operador) — grava quem assumiu e a hora; também atualiza o status do pedido para "em_preparo" via PedidoDAO.
Botão "Concluir" (em cada card de pedido da fila)	onclick abre modal com acao=concluir pré-selecionada	POST /app/fila (acao=concluir)	FilaController.concluirPreparo()	FilaPreparoDAO.concluir(id) — grava data_conclusao; atualiza o status do pedido para "pronto".
Botão "🔄 Atualizar"	onclick="location.reload()"	GET /app/fila	FilaController.doGet()	Recarrega as 3 filas do zero (FilaPreparoDAO.listarFila por setor).
Botão "Confirmar" (dentro do modal de assumir/concluir)	Envia <form id="formModal">	POST /app/fila (acao definida no campo oculto #modalAcao)	FilaController.doPost()	Roteia para iniciarPreparo() ou concluirPreparo() conforme a ação escolhida.

8. Área do Cliente (perfil USUARIO)

Todas as telas a seguir são exclusivas do perfil USUARIO (cliente final do app) e ficam sob o ClienteController — GERENTE e FUNCIONARIO não usam estas rotas.

8.1 Pedido Direto na Mesa

Arquivo(s) JSP: WEB-INF/views/cliente/mesa.jsp

Controller: ClienteController

DAO(s) envolvidas: MesaDAO, CardapioDAO, PedidoDAO, ItemPedidoDAO, FilaPreparoDAO

Quem pode acessar: Exclusivo USUARIO

Tela pensada para o cliente sentado à mesa (ex.: acessando via QR code): pode chamar o garçom ou já montar o próprio pedido.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Botão "🔔 Chamar Garçom"	Envia <form method="POST" action="/app/cliente/mesa"> com acao=chamarGarcom	POST /app/cliente/mesa (acao=chamarGarcom)	ClienteController – bloco chamarGarcom	Sinaliza a mesa como "chamado pendente" (visto pelo funcionário em MesaController — botão "Atender chamado", seção 5.1).
Botão "📝 Fazer Pedido Direto"	onclick="irParaPedidoDireto()" (mostra a seção de itens na mesma página)	– (JS local)	–	Só exibe o formulário de itens; ainda não chama o servidor.
Botões +/- de quantidade por item	onclick="diminuir()" / "aumentar()"	– (JS, monta campos ocultos)	–	Monta as listas paralelas itemId/quantidade que serão enviadas no submit.
Botão "Confirmar Pedido" (submit)	Envia <form method="POST"> com acao=pedidoDireto	POST /app/cliente/mesa (acao=pedidoDireto)	ClienteController – bloco pedidoDireto	Cria o Pedido vinculado à mesa (tipo=mesa), sempre usando o preço ATUAL do cardápio; insere itens e cria a entrada na fila de preparo — mesma lógica de criação usada no delivery.

8.2 Delivery

Arquivo(s) JSP: WEB-INF/views/cliente/delivery.jsp

Controller: ClienteController

DAO(s) envolvidas: CardapioDAO, PedidoDAO, ItemPedidoDAO, FilaPreparoDAO

Quem pode acessar: Exclusivo USUARIO

Pedido feito pelo próprio cliente para entrega (não vinculado a nenhuma mesa).

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Pills de filtro por setor	onclick="filtrar(...)"	– (JS, filtro local)	–	Filtra os itens exibidos, sem requisição.
Botões +/- de quantidade por item	onclick	– (JS)	–	Monta as listas paralelas itemId/quantidade.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Botão "Confirmar Pedido" (submit)	Envia <form method="POST" action=".../app/cliente/delivery">	POST /app/cliente/delivery	ClienteController.confirmarDelivery()	Cria Pedido (tipo=delivery, identificador_operador = nome do cliente logado), valida disponibilidade de cada item no cardápio, usa o preço atual, insere itens em lote e cria a entrada na fila de preparo. Tudo em transação (rollback se nenhum item for válido).
Link "Meus Pedidos"	Navegação <a href>	GET /app/cliente/meus-pedidos	ClienteController.exibirMeusPedidos()	Leva para a seção 8.4.

8.3 Reserva de Mesa

Arquivo(s) JSP: WEB-INF/views/cliente/reserva.jsp

Controller: ClienteController + MesaController (mesma tabela mesa)

DAO(s) envolvidas: MesaDAO

Quem pode acessar: Exclusivo USUARIO

Lista as mesas livres para o cliente escolher e reservar.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Botão "Reservar" (por mesa livre, submit)	Envia <form method="POST" action=".../app/cliente/reserva">	POST /app/cliente/reserva	ClienteController – bloco confirmarReserva (delega a mudança de status a MesaController.mudarStatus)	Confere que a mesa está "livre"; atualiza status para "reservada" via MesaDAO, gravando o nome do cliente como operador.

8.4 Meus Pedidos

Arquivo(s) JSP: WEB-INF/views/cliente/meus_pedidos.jsp

Controller: ClienteController

DAO(s) envolvidas: PedidoDAO, ItemPedidoDAO

Quem pode acessar: Exclusivo USUARIO

Histórico de pedidos do cliente logado (identificado pelo nome, gravado em identificador_operador).

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Link "🔄 Atualizar agora"	Navegação <a>	GET /app/cliente/meus-pedidos	ClienteController.exibirMeusPedidos()	PedidoDAO.listarPorOperador(nomeDoCliente) + carrega os itens de cada pedido para exibir o detalhe.
Link "Fazer novo pedido" / "Peça agora"	Navegação <a href>	GET /app/cliente/delivery	ClienteController.exibirDelivery()	Leva de volta para a tela de delivery (seção 8.2).

9. Gestão de Funcionários (Staff)

9.1 Listagem de Funcionários/Gerentes

Arquivo(s) JSP: WEB-INF/views/staff/staff.jsp

Controller: StaffController

DAO(s) envolvidas: UsuarioDAO

Quem pode acessar: Exclusivo GERENTE

Diferente do autocadastro público (seção 2.2), aqui o GERENTE cria contas internas com perfil FUNCIONARIO ou GERENTE.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Botão "+ Nova Conta"	Navegação <a href>	GET /app/staff?acao=novo	StaffController.exibirNovo()	Abre form_staff.jsp em branco.
Ícone lixeira "Desativar" (por funcionário)	Envia <form method="POST"> oculto com acao=desativar	POST /app/staff (acao=desativar)	StaffController.desativar()	UsuarioDAO.desativar(id) — soft delete, com duas proteções: não permite desativar a própria conta, e não permite desativar o último GERENTE ativo do sistema.

9.2 Formulário de Nova Conta

Arquivo(s) JSP: WEB-INF/views/staff/form_staff.jsp

Controller: StaffController

DAO(s) envolvidas: UsuarioDAO

Quem pode acessar: Exclusivo GERENTE

Cadastro de uma nova conta interna, com escolha explícita de perfil e função.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Select "Perfil" (Funcionário/Gerente)	Seleção	—	—	Só pode ser FUNCIONARIO ou GERENTE — nunca USUARIO (validado também no servidor).
Select "Função" (Atendente/Cozinha)	Seleção — só relevante quando perfil=FUNCIONARIO	—	—	Campo informativo (funcao); desde a v3 não muda mais as permissões de acesso, apenas identifica o papel do funcionário.
Botão "Criar conta" (submit)	Envia <form method="POST" action=".../app/staff" onsubmit="return validar()"> com acao=salvar	POST /app/staff (acao=salvar)	StaffController.salvar()	Valida perfil (só FUNCIONARIO/GERENTE), gera hash BCrypt da senha e insere via UsuarioDAO.inserir().
Link "Cancelar" / "← Voltar"	Navegação <a href>	GET /app/staff	StaffController.listar()	Volta para a listagem sem salvar.

10. Relatórios em PDF

10.1 Geração de Relatório

Arquivo(s) JSP: WEB-INF/views/relatorio/relatorio.jsp

Controller: RelatorioController

DAO(s) envolvidas: PedidoDAO, ItemPedidoDAO + utilitário RelatorioPDF

Quem pode acessar: Exclusivo GERENTE

Tela de seleção de período que gera um PDF de faturamento para download.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Radio buttons "Hoje" / "Semana" / "Mês" / "Customizado"	onchange="selecionarPeriodo(...)"	– (JS mostra/esconde os campos de data customizada)	–	Define o valor do parâmetro periodo que será enviado no POST.
Campos de data (quando "Customizado")	Digitação/seleção de data	–	–	Preenche dataInicio/dataFim (formato yyyy-MM-dd).
Botão "Gerar Relatório" (#btnGerar, submit)	Envia <form method="POST" action=".../app/relatorios">	POST /app/relatorios	RelatorioController.doPost()	Resolve o intervalo de datas conforme periodo (hoje/semana/mes/customizado, com fallback pro mês corrente em caso de erro), busca pedidos entregues no período (buscarPedidosEntregues), agrupa faturamento por categoria (agruparPorCategoria) e chama RelatorioPDF.gerar() — a resposta é enviada como download (Content-Disposition: attachment).

11. Navegação Global e Componentes Auxiliares

11.1 Menu Lateral (Sidebar)

Arquivo(s) JSP: WEB-INF/views/shared/_sidebar.jsp (incluído em todas as telas internas)

Controller: N/A – apenas links de navegação para os Controllers já listados

Quem pode acessar: Itens exibidos variam conforme o perfil logado (GERENTE vê mais itens que FUNCIONARIO/USUARIO)

Fragmento JSP compartilhado (padrão "include") que monta o menu conforme o perfil da sessão.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Link "Dashboard"	Navegação	GET /app/dashboard	DashboardController	Só visível para GERENTE.
Link "Mesas"	Navegação	GET /app/mesas	MesaController.listarMesas()	Visível para GERENTE/FUNCIONARIO.
Link "Cardápio"	Navegação	GET /app/cardapio	CardapioController.listar()	Visível para todos os perfis (com regras de escrita diferentes).
Link "Pedidos"	Navegação	GET /app/pedidos	PedidoController.listar()	Visível para GERENTE/FUNCIONARIO.
Link "Fila"	Navegação	GET /app/fila	FilaController.doGet()	Visível para GERENTE/FUNCIONARIO.
Link "Relatórios"	Navegação	GET /app/relatorios	RelatorioController.doGet()	Só visível para GERENTE.
Link "Staff"	Navegação	GET /app/staff	StaffController.listar()	Só visível para GERENTE.
Links "Minha Mesa" / "Delivery" / "Reservar" / "Meus Pedidos"	Navegação	GET /app/cliente/*	ClienteController	Só visíveis para USUARIO (cliente).
Botão 🌙 (alternar tema claro/escuro)	onclick="alternarTema()"	– (JS + localStorage do navegador)	–	Não passa pelo servidor; só altera classe CSS/preferência salva no navegador.
Link "Sair"	Navegação	GET /auth/logout	AuthController.logout()	Ver seção 2.3.

11.2 Imagens do Cardápio

Arquivo(s) JSP: Usado dentro de cardapio.jsp e form_item.jsp: ``

Controller: ImagemServlet

DAO(s) envolvidas: – (usa UploadImagemUtil para localizar a pasta de uploads)

Quem pode acessar: Pública (não passa pelo AuthFilter, que só protege /app/*)

As imagens dos itens são salvas FORA da pasta do projeto (para sobreviver a um "clean/republish" do Tomcat), então o próprio Tomcat não consegue servi-las como um arquivo estático comum — este servlet lê o arquivo do disco e devolve como resposta HTTP.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Tag exibindo a foto do item	Requisição de imagem feita pelo navegador	GET /imagens/cardapio/{nomeArquivo}	ImagemServlet.doGet()	Resolve o caminho do arquivo dentro da pasta de uploads (com proteção contra path traversal — rejeita qualquer caminho que tente "escapar" da pasta), define o Content-Type pela extensão e copia os bytes do arquivo para a resposta, com cache de 1 dia.

12. Resumo por Camada (Controllers, DAOs, Models)

12.1 Controllers (br.com.restaurante.controller)

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
AuthController	/auth/login, /auth/cadastro, /auth/logout	—	UsuarioDAO	Login, cadastro público e logout.
DashboardController	/app/dashboard	GERENTE	MesaDAO, PedidoDAO, ItemPedidoDAO	Painel gerencial (só leitura).
CardapioController	/app/cardapio	leitura: todos / escrita: GERENTE	CardapioDAO, CategoriaItemDAO	CRUD de itens e categorias do cardápio.
MesaController	/app/mesas	GERENTE/FUNCIONAR IO (+ reserva por USUARIO)	MesaDAO, PedidoDAO	Ciclo de vida das mesas.
PedidoController	/app/pedidos	GERENTE/FUNCIONAR IO	PedidoDAO, ItemPedidoDAO, MesaDAO, CardapioDAO, FilaPreparoDAO, PagamentoDAO	Criação e acompanhamento de pedidos + pagamento.
FilaController	/app/fila	GERENTE/FUNCIONAR IO	FilaPreparoDAO, PedidoDAO	Fila de preparo por setor.
RelatorioController	/app/relatorios	GERENTE	PedidoDAO, ItemPedidoDAO, RelatorioPDF	Geração de relatório em PDF.
StaffController	/app/staff	GERENTE	UsuarioDAO	Cadastro de contas internas.
ClienteController	/app/cliente/*	USUARIO	PedidoDAO, ItemPedidoDAO, MesaDAO, CardapioDAO, FilaPreparoDAO	Delivery, reserva, pedido na mesa e histórico.
ImagemServlet	/imagens/cardapio/ *	pública	UploadImagemUtil	Serve as fotos dos itens do cardápio.

12.2 DAOs (br.com.restaurante.dao) — acesso ao banco

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
UsuarioDAO	buscarPorLogin / buscarPorId / listarStaff / inserir / editar / desativar	—	tabela usuario	Usado por AuthController e StaffController.
CardapioDAO	listar / listarPorCategoria / buscarPorId / inserir / editar / atualizarDisponibili dade / desativar	—	tabela cardapio	Usado por CardapioController, PedidoController, ClienteController.
CategoriaItemDAO	listar / listarPorSetor / buscarPorId / inserir / editar / desativar	—	tabela categoria_item	Usado por CardapioController.

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
MesaDAO	listar / listarLivres / buscarPorId / inserir / editar / atualizarStatus / chamarGarcom / atenderChamado / desativar	—	tabela mesa	Usado por MesaController, PedidoController, ClienteController, DashboardController.
PedidoDAO	listarAbertos / listarPorMesa / listarPorStatus / listarPorOperador / buscarPorId / inserir / editar / atualizarStatus / desativar	—	tabela pedido	Usado por PedidoController, ClienteController, DashboardController, RelatorioController.
ItemPedidoDAO	inserir / inserirLote / listarPorPedido / buscarPorId / atualizarStatus / desativar / cancelarItensDoPedido	—	tabela item_pedido	Usado por PedidoController, ClienteController.
FilaPreparoDAO	listarFila / listarFilaGeral / buscarPorPedido / buscarPorId / inserir / iniciarPreparo / concluir / desativar	—	tabela fila_preparo	Usado por FilaController, PedidoController, ClienteController.
PagamentoDAO	registrar / listarPorPedido / somarPorPedido	—	tabela pagamento	Usado por PedidoController ao marcar pedido como entregue.

12.3 Utilitários (br.com.restaurante.util)

Elemento na tela	Ação / evento	Rota (HTTP)	Classe.método (Java)	O que faz
Conexao	Fornece Connection ao banco (JDBC)	—	Usado por todos os DAOs e Controllers.	Lê configurações de db.properties.
UploadImagemUtil	Salva/gera nome único (UUID) para as imagens do cardápio	—	Usado por CardapioController.	Grava fora da pasta do projeto (sobrevive a clean/republish).
RelatorioPDF	Monta o PDF de faturamento	—	Usado por RelatorioController.	Gera o arquivo final entregue como download.
AuthFilter (filter)	Intercepta toda rota /app/*	—	Aplica-se a todos os Controllers internos.	Bloqueia acesso sem sessão ativa (usuarioLogado).
CharsetFilter (filter)	Intercepta todas as requisições	—	Global.	Garante encoding UTF-8 em request/response.